C++ COMPLETE REFERENCE LIBRARY • MODULE 09

09 CLASSES AND OBJECTS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 classroom person and encapsulation
3. **Theory Chapter:** 02 this pointer and const member functions
4. **Theory Chapter:** 03 static members and object arrays
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (4 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

09 — Classes and Objects

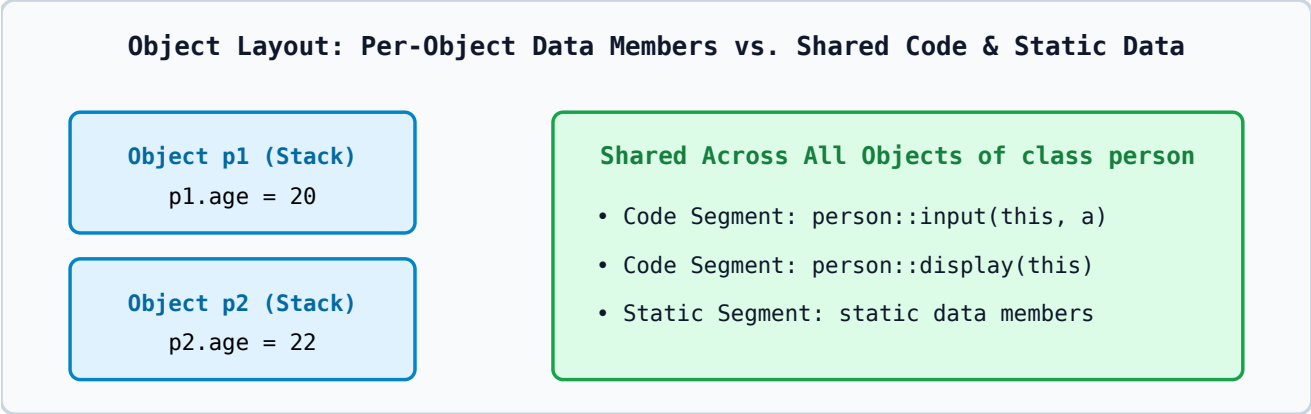
This module covers class declarations, data encapsulation (`private` , `protected` , `public`), the baseline classroom `person` example, the implicit `this` pointer, `const` member functions, `static` data members and member functions, and arrays of objects.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_classroom_person_and_encapsulation.md` (./01_THEORY/01_classroom_person_and_encapsulation.md)
  - `02_this_pointer_and_const_member_functions.md` (./01_THEORY/02_this_pointer_and_const_member_functions.md)
  - `03_static_members_and_object_arrays.md` (./01_THEORY/03_static_members_and_object_arrays.md)
- **Programs (`02_PROGRAMS/`)**:

  - `01_classroom_person_basic.cpp` (./02_PROGRAMS/01_classroom_person_basic.cpp) (Category A — Exact Uploaded Classroom Baseline Program)
  - `02_private_access_compile_error.cpp` (./02_PROGRAMS/02_private_access_compile_error.cpp) (Category B)
  - `03_this_pointer_and_const_methods.cpp` (./02_PROGRAMS/03_this_pointer_and_const_methods.cpp) (Category A)
  - `04_static_member_counter.cpp` (./02_PROGRAMS/04_static_member_counter.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`) & Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_class_object_memory_layout.svg` (./05_DIAGRAMS/diagram_01_class_object_memory_layout.svg)

01 — Classes, Objects, Encapsulation, and the Baseline Classroom `person` Program



Class Object Memory Layout

1. The Baseline Classroom `person` Program

CPP

```cpp
#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    person p;

    p.input(20);
    p.display();

    return 0;
}
```

Line-by-Line Execution and Memory Walkthrough

1. **Class Definition (`class person`)**: Defines a blueprint (consumes 0 bytes of runtime memory until an object is instantiated).
2. **`private: int age;`**: Hides `age` so outside functions like `main()` cannot write `p.age = -5;`.
3. **`public: void input(int a)` and `void display()`**: Public interface functions that can read and modify `age`.
4. **`person p;` in `main()`**: Allocates `sizeof(person)` (`4` bytes for `int age`) on the stack frame of `main()`.
5. **`p.input(20);`**: Calls `person::input` with hidden `this = &p` and sets `p.age = 20`.
6. **`p.display();`**: Reads `p.age` (`20`) and prints `Age: 20`.

2. Difference Between `class` and `struct` in C++

In C++, `struct` and `class` are identical in capabilities (both can have constructors, member functions, inheritance, and access specifiers) with only **two default differences**:

1. Default member access is `private` in a `class`, and `public` in a `struct`.
2. Default inheritance mode is `private` for a `class`, and `public` for a `struct`.

02 — The `this` Pointer and `const` Member Functions

1. What is the `this` Pointer?

Every non-static member function receives an implicit hidden pointer named `this` holding the address of the object on which the member function was called:

- Inside `void person::input(int age)`, if the parameter name `age` shadows the member variable `age`, you write `this→age = age;`.
- Returning `*this` by reference (`person&`) enables **method chaining** (`p.set_age(20).display()`).

2. `const` Member Functions (`void display() const`)

Placing `const` after the parameter list changes the type of `this` from `person* const` to `const person* const`:

- Guarantees the member function will **not** modify any non-`mutable` data member of the object.
- **Crucial Rule**: A `const` object (`const person p;`) can **ONLY** call `const` member functions! Calling a non-`const` member function on a `const` object is a compilation error.

03 — `static` Data Members, `static` Member Functions, and Object Size

1. `static` Data Members

A `static` data member is shared by **all** objects of the class (stored once in static memory, not inside each object):

- In C++17 / C++20 / C++23, you can initialize a `static` data member right inside the class using `inline static int count = 0;`.
- Because `static` data members are not part of individual objects, they do **not** contribute to `sizeof(MyClass)`.

2. `static` Member Functions

- Can be called using the class name without any object: `person::show_count();`.
- Have **no** `this` **pointer** and therefore **cannot** access non-static data members (`age`) or call non-static member functions directly!

Visual Concept & Memory Diagrams (09 CLASSES AND OBJECTS)

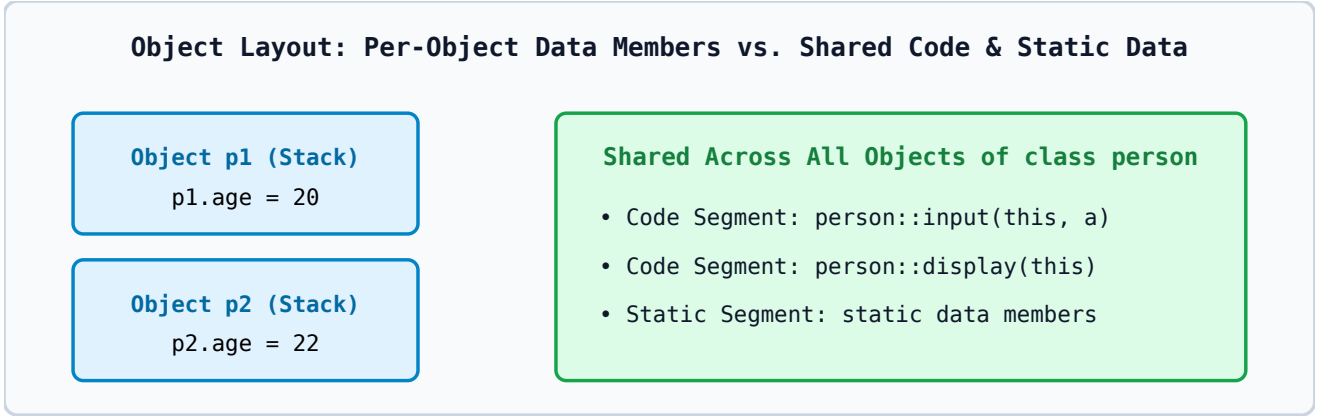


Diagram: diagram 01 class object memory layout

Annotated C++ Programs (09 CLASSES AND OBJECTS)

Program: `01_classroom_person_basic.cpp`

C++ SOURCE FILE • 01_classroom_person_basic.cpp

```cpp
// Program Title: Baseline Classroom 'person' Class Demonstration
// Topic: Classes and Objects — Encapsulation and Member Functions
// Purpose: Preserve the foundational classroom person class with private age, input(int),
and display().
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_classroom_person_and_encapsulation.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    person p;

    p.input(20);
    p.display();

    return 0;
}

/*
Expected Deterministic Output:
Age: 20
*/
```

Program: `02_private_access_compile_error.cpp`

C++ SOURCE FILE • 02_private_access_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Direct Access to Private Data Member From main()
// Topic: Classes and Objects — Access Specifiers
// Purpose: Demonstrate why accessing private member 'p.age' directly inside main() fails to compile.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_classroom_person_and_encapsulation.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }
};

int main() {
    person p;
    p.age = 20; // COMPILE ERROR: 'int person::age' is private within this context
    return 0;
}

/*
Why Compilation Fails:
'age' is declared under the 'private:' access specifier in 'class person'.
Non-member functions like 'main()' cannot access private members directly; use 'p.input(20);' as shown in 01_classroom_person_basic.cpp.
*/
```

Program: `03_this_pointer_and_const_methods.cpp`

C++ SOURCE FILE • 03_this_pointer_and_const_methods.cpp

```cpp
// Program Title: Using the 'this' Pointer and 'const' Member Functions
// Topic: Classes and Objects — this Pointer & const Correctness
// Purpose: Show resolving parameter shadowing via 'this→age = age' and calling a const
display() on a const object.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_this_pointer_and_const_member_functions.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    person(int age = 0) {
        this→age = age;
    }

    void display() const {
        cout << "Const-safe Age: " << this→age << endl;
    }
};

int main() {
    const person p(21);
    p.display();
    return 0;
}

/*
Expected Deterministic Output:
Const-safe Age: 21
*/
```

Program: `04_static_member_counter.cpp`

C++ SOURCE FILE • 04_static_member_counter.cpp

```cpp
// Program Title: Counting Active Objects Using Static Data Member and Static Function
// Topic: Classes and Objects — static Members
// Purpose: Demonstrate shared static data member and static member function in a classroom student class.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_static_members_and_object_arrays.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class student {
private:
    int roll;
    inline static int total_students = 0;

public:
    student(int r) {
        roll = r;
        total_students++;
    }

    static void show_total() {
        cout << "Total students created: " << total_students << endl;
    }
};

int main() {
    student s1(101);
    student s2(102);
    student::show_total();
    return 0;
}

/*
Expected Deterministic Output:
Total students created: 2
*/
```

Conceptual Questions — Module 09: Classes and Objects

Q1. What is the size of an empty class (`class Empty {};`) in C++, and why is it never `0` bytes? **Q2.** Why do `static` member functions not have a `this` pointer?

Debugging Questions — Module 09

Q4. Why does `const person p; p.display();` fail to compile if `display()` is declared as `void display()` without `const`?

Predict the Output Questions — Module 09

Q3. What is the output of `sizeof(person)` when `person` contains `int age;` and `static int count;`?

Programming Questions — Module 09

Q5. Write a C++23 classroom program defining `class employee` with `id`, `salary`, `input()`, and `display()` for an array of 3 employees.

Detailed Solutions — Conceptual Questions (Module 09)

- **Q1 Solution**: `sizeof(Empty)` is at least `1` byte so that two distinct objects of type `Empty` (e.g., in an array `Empty arr[2]`) are guaranteed by the C++ standard to have distinct memory addresses (`&arr[0]` ≠ `&arr[1]`).
- **Q2 Solution**: A `static` member function belongs to the class as a whole and can be invoked without any object instance (`MyClass::func()`), so there is no invoking object for `this` to point to.

Detailed Solutions — Predict the Output & Debugging (Module 09)

- **Q3 Solution**: `sizeof(person)` equals `sizeof(int)` (typically `4` bytes), because `static int count` is stored once in static storage and is not part of individual `person` object instances.
- **Q4 Solution**: Calling a non-`const` member function on `const person p` attempts to pass `const person*` as the implicit `person* const this` parameter, which violates `const` qualification. Fix: declare `void display() const`.

Detailed Solutions — Programming Questions (Module 09)

CPP

```cpp
#include <iostream>
using namespace std;

class employee {
private:
    int id;
    int salary;
public:
    void input(int i, int s) { id = i; salary = s; }
    void display() const { cout << "ID: " << id << ", Salary: " << salary << endl; }
};

int main() {
    employee emp[3];
    emp[0].input(1, 50000);
    emp[1].input(2, 60000);
    emp[2].input(3, 70000);
    for (int i = 0; i < 3; i++) emp[i].display();
    return 0;
}
```